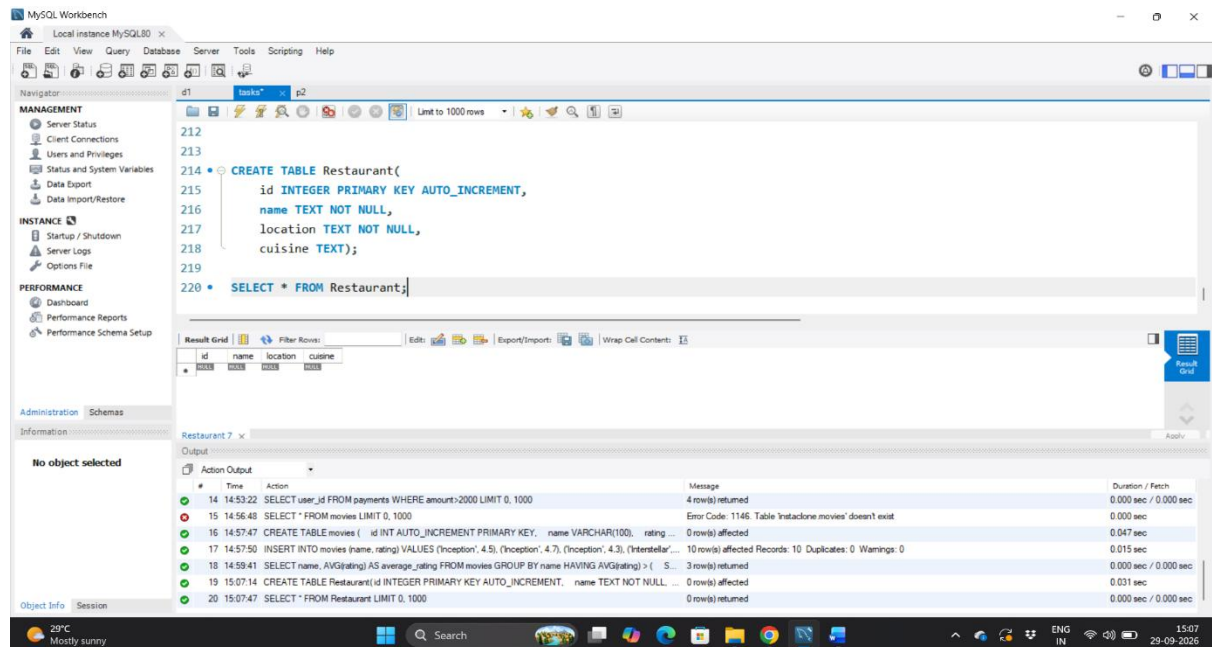
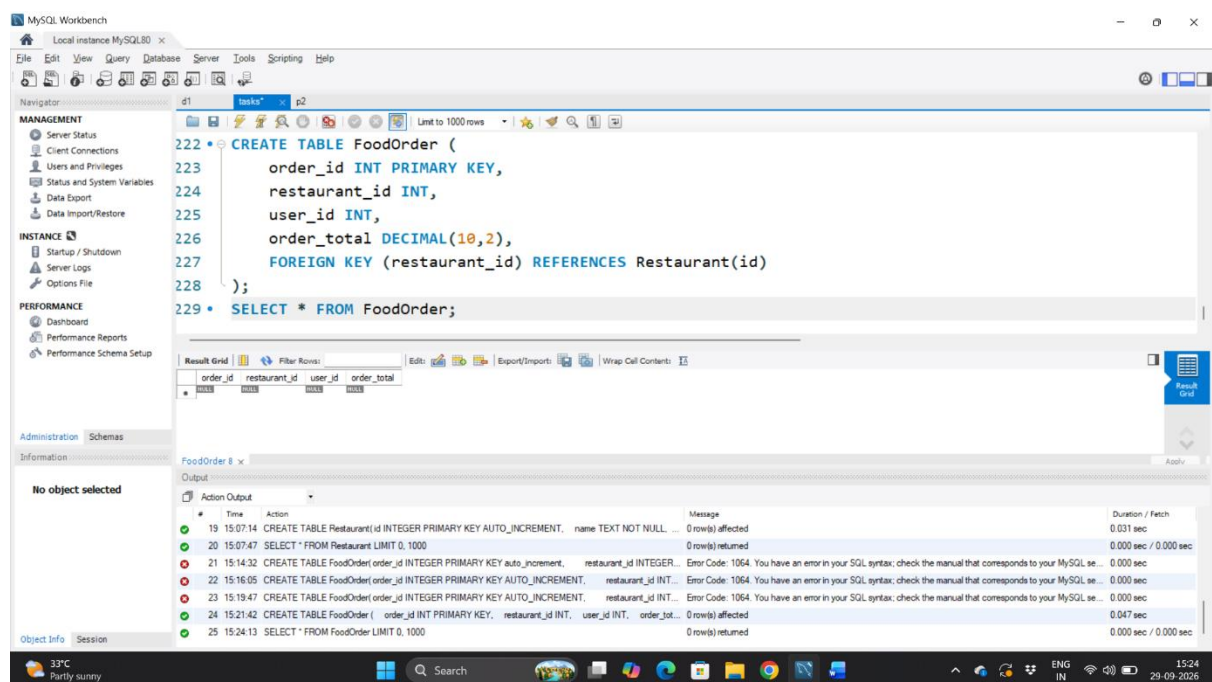


Session 7

Q1. Create a table called Restaurant with columns: id, name, location, and cuisine. Apply NOT NULL constraint to name and location, and make id the PRIMARY KEY.



Q2. Create a table called FoodOrder with columns: order_id, restaurant_id, user_id, and order_total. Set order_id as PRIMARY KEY, and add a FOREIGN KEY constraint on restaurant_id referencing Restaurant(id).



Q3. Insert 6 rows into a table called SpotifyUser with columns: user_id, username, email, and subscription_type. Ensure that username is UNIQUE and email is NOT NULL. Hint Use the UNIQUE and NOT NULL constraints when creating the table.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```

231 CREATE TABLE spotifyuser(
232     user_id INT PRIMARY KEY,
233     username VARCHAR(100) UNIQUE,
234     email VARCHAR(100) NOT NULL,
235     subscription_type VARCHAR(100));
236 INSERT INTO spotifyuser (user_id, username, email, subscription_type)
237 VALUES
238 (1, 'musiclover01', 'musiclover01@gmail.com', 'Premium'),
239 (2, 'rockfan22', 'rockfan22@gmail.com', 'Free'),
240 (3, 'popstar99', 'popstar99@gmail.com', 'Premium'),
241 (4, 'beatmaster', 'beatmaster@gmail.com', 'Student'),
242 (5, 'melodyking', 'melodyking@gmail.com', 'Premium'),
243 (6, 'songlover55', 'songlover55@gmail.com', 'Free');
244 SELECT * FROM SpotifyUser;
    
```

The Result Grid shows the following data:

#	user_id	username	email	subscription_type
1	1	musiclover01	musiclover01@gmail.com	Premium
2	2	rockfan22	rockfan22@gmail.com	Free
3	3	popstar99	popstar99@gmail.com	Premium
4	4	beatmaster	beatmaster@gmail.com	Student
5	5	melodyking	melodyking@gmail.com	Premium

The Action Output shows the execution of the SQL statements, including the creation of the table and the insertion of 6 rows.

Q4. Create a VIEW named TopSpendersView that shows usernames and order_total from a FoodOrder table where order_total is greater than 1000.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following code:

```

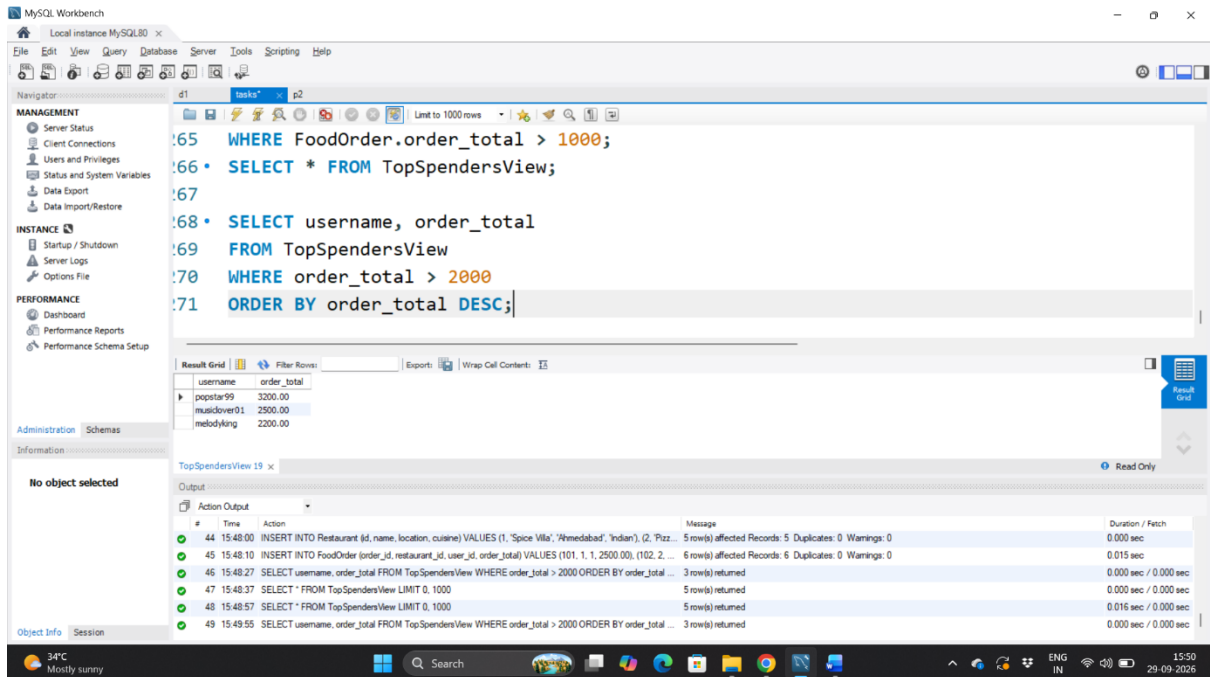
258 SELECT * FROM SpotifyUser;
259
260 CREATE VIEW TopSpendersView AS
261 SELECT SpotifyUser.username, FoodOrder.order_total
262 FROM SpotifyUser
263 INNER JOIN FoodOrder
264 ON SpotifyUser.user_id = FoodOrder.user_id
265 WHERE FoodOrder.order_total > 1000;
266 SELECT * FROM TopSpendersView;
267
    
```

The Result Grid shows the following data:

#	username	order_total
1	musiclover01	2500.00
2	rockfan22	1500.00
3	popstar99	3200.00
4	melodyking	2200.00
5	songlover55	1200.00

The Action Output shows the execution of the SQL statements, including the creation of the view and the selection of data from the view.

Q5. Write a SQL query to list all users from the TopSpendersView who have spent more than 2000, sorted by order_total descending. Constraint: Only use the TopSpendersView for your query, not the original FoodOrder table.



The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```
65 WHERE FoodOrder.order_total > 1000;  
66 • SELECT * FROM TopSpendersView;  
67  
68 • SELECT username, order_total  
69 FROM TopSpendersView  
70 WHERE order_total > 2000  
71 ORDER BY order_total DESC;
```

The Results Grid shows the output of the query:

username	order_total
pupatar99	3200.00
musiclover01	2500.00
melodyking	2200.00

The Action Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
44	15:48:00	INSERT INTO Restaurant (id, name, location, cuisine) VALUES (1, 'Spice Villa', 'Mumbai', 'Indian'), (2, 'Pizz...	5 row(s) affected Records: 5 Duplicates: 0 Warnings: 0	0.000 sec
45	15:48:10	INSERT INTO FoodOrder (order_id, restaurant_id, user_id, order_total) VALUES (101, 1, 1, 2500.00), (102, 2, ...	6 row(s) affected Records: 6 Duplicates: 0 Warnings: 0	0.015 sec
46	15:48:27	SELECT username, order_total FROM TopSpendersView WHERE order_total > 2000 ORDER BY order_total ...	3 row(s) returned	0.000 sec / 0.000 sec
47	15:48:37	SELECT * FROM TopSpendersView LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
48	15:48:57	SELECT * FROM TopSpendersView LIMIT 0, 1000	5 row(s) returned	0.016 sec / 0.000 sec
49	15:49:55	SELECT username, order_total FROM TopSpendersView WHERE order_total > 2000 ORDER BY order_total ...	3 row(s) returned	0.000 sec / 0.000 sec